

The TRINE Signature Scheme: Algorithm Specifications and Supporting Documentation

Gang Tang^{1,2}, Debiao He³, Yu Dai⁷, Cong Peng², Min Luo², Yinan Li^{4,5,6}, Xinyi Huang⁸, Runqing Xu², Zihao Yu², and Bingbing Xia⁹

¹ Institute for Math & AI, Wuhan, Wuhan, China

² School of Cyber Science and Engineering, Wuhan University, Wuhan, China

³ School of Computer Science, Wuhan University, Wuhan, China

⁴ School of Artificial Intelligence, Wuhan University, Wuhan, China

⁵ Hubei Center for Applied Mathematics, Wuhan, China

⁶ Hubei Key Laboratory of Computational Science, Wuhan, China

⁷ School of Mathematics and Statistics, Wuhan University of Technology, Wuhan, China

⁸ College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China

⁹ Beijing Certificate Authority Co. Ltd, Beijing, China

Table of Contents

1	Introduction	3
1.1	Overview of the basic approach	4
1.2	Advantages and limitations	5
2	Basic operations	5
2.1	Basic notations	5
2.2	Trilinear forms and a natural group action on them	6
2.3	TFE, matrix code equivalence and tensor isomorphism	6
2.4	Corank-one points and the canonical form	6
3	The TRINE signature scheme	11
3.1	Parameters	11
3.2	Key generation	12
3.3	Signature generation and verification	12
3.4	Unbalanced challenge space	13
3.5	The formulas for public key, private key, and signature sizes	14
4	Performance analysis	14
4.1	Key and signature sizes	15
4.2	Performance	15
5	Security reductions	16
5.1	Assumptions	16
5.2	Security of TRINE	16
5.3	Algorithms of the TFE problem	17
6	Parameter choices	20
6.1	The choices of n	21
6.2	The choices of q	21
6.3	The choices of X, r and K	22

1 Introduction

We present the digital signature scheme **TRINE** based on the hardness of the Trilinear Form Equivalence (TFE) problem.

The construction follows the Goldreich–Micali–Wigderson (GMW) [GMW91] paradigm, from which we derive a zero-knowledge identification protocol for the TFE problem. Instead of working directly with full algebraic witnesses, our protocol uses a new viewpoint based on corank-one invariants, inspired by recent work proposed by Narayanan, Qiao and Tang [NQT24]. In this approach, the prover associates each instance with an invariant derived from a chosen corank-one point of the underlying trilinear form. The verifier only checks consistency of these invariants across commitments, rather than reconstructing the full equivalence. We then apply the Fiat–Shamir (FS) [FS86] transformation to remove the interaction from the zero-knowledge protocol and obtain a digital signature scheme. An overview of these steps will be presented in Section 1.1

Some key considerations for **TRINE** are as follows:

Relation to NIST signature schemes. **TRINE** belongs to the same broad family of Fiat–Shamir signatures from isomorphism and equivalence problems as **LESS** [BMPS20], **MEDS** [CNP⁺23a,CNP⁺23b], and **ALTEQ** [BDN⁺23,TDJ⁺22]. **MEDS** and **ALTEQ** were submitted to the first round of NIST’s call for additional post-quantum digital signatures. **MEDS** is based on Matrix Code Equivalence (MCE), while **ALTEQ** is based on Alternating Trilinear Form Equivalence (ATFE). **LESS** is based on linear code equivalence and was selected as a second-round candidate in the same NIST process.

These underlying problems are closely related from the viewpoint of tensor isomorphism. Matrix Code Equivalence can be represented as a trilinear form equivalence problem by taking a basis of the matrix code as tensor slices. Conversely, a cubic trilinear form can be viewed as a three-way tensor and, for generic instances with linearly independent slices, gives rise to a matrix code instance. Thus the problem underlying **MEDS** and the TFE problem used in **TRINE** are different presentations of the same tensor isomorphism framework. Moreover, MCE and ATFE are known to be polynomial-time equivalent and Tensor-Isomorphism complete [GQT22]. In this sense, the hard problems underlying **MEDS**, **ALTEQ**, and **TRINE** lie in the same polynomial-equivalence class of isomorphism problems. The problem underlying **LESS** is also connected to this framework. **LESS** is based on linear code equivalence in the Hamming metric, and the corresponding monomial code-equivalence problem reduces in polynomial time to tensor isomorphism [GQ23], and hence to the general TFE framework.

The main design difference is in the response representation. **MEDS** and **ALTEQ** follow the standard GMW-style approach in which a response contains one or more invertible linear transformations. **TRINE** instead uses the corank-one invariant: the prover opens an effective corank-one point, and the verifier recomputes the corresponding canonical form. This replaces matrix-valued responses by vector-valued responses, reducing the communication per non-base challenge from roughly $O(n^2 \log q)$ bits to roughly $n \log q$ bits.

Support for TFE: post-quantum, complexity-theoretic, and practical.

1. From the quantum algorithm viewpoint, there is a strong negative evidence for the standard approach of the hidden subgroup problems to work for general linear groups over finite fields [HMR⁺10]. This could be considered as an inherent bottleneck for

current quantum algorithms to work against the general linear hidden subgroup problem. Such an argument is applicable to TFE as it relies on a group action by general linear groups over finite fields.

2. From the computational complexity viewpoint, TFE is a natural form of the 3-tensor isomorphism problem. A trilinear form is equivalently represented by a 3-way tensor, and the TFE action is precisely the natural $GL \times GL \times GL$ action on the 3-tensor. The modern complexity theory of tensor isomorphism shows that tensor, algebra, polynomial, and group isomorphism problems are connected, and many such problems are polynomial-time equivalent [GQ23,GQ24,GQT22]. Matrix Code Equivalence can also be represented as TFE by stacking a basis of a matrix code as slices of a trilinear form; conversely, a random cubic TFE instance gives rise, to a matrix code instance through its slices.
3. From the practical algorithm viewpoint, the connections with other problems allows us to tap into the pool of practical algorithms for polynomial isomorphism [BFV13], matrix code equivalence [CNP⁺23b], group isomorphism [LQ17], and Min-rank problem [BBC⁺20]. The TRINE parameters take into account the currently most relevant attacks, including the corank-one invariant attack of Narayanan–Qiao–Tang [NQT24], the hull-based matrix-code attack of Couvreur–Levrat [CL25], and the rare-triangle algebraic attack of Ran–Samardjiska [RS24].

1.1 Overview of the basic approach

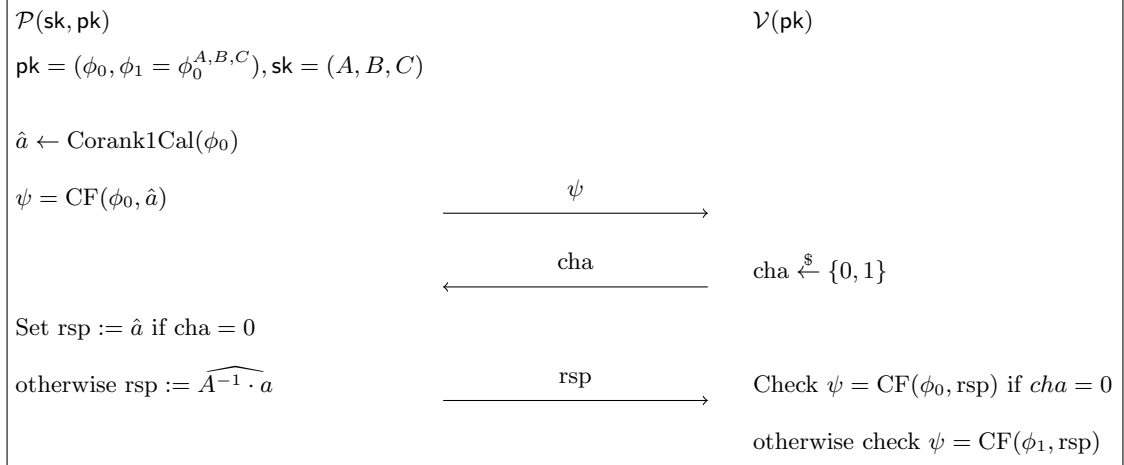


Fig. 1: The Basic Sigma protocol of TRINE.

This protocol consists of two steps. First, apply the Goldreich–Micali–Wigderson (GMW) protocol to TFE to obtain an identification (or Sigma) protocol¹⁰. Second, apply the Fiat–Shamir (FS) transformation to the identification protocol.

The resulting Sigma protocol is illustrated in Fig. 1. Let $\phi_0, \phi_1 \in \text{TFE}(n, q)$ be two equivalent trilinear forms, and let the secret key contain an equivalence matrix A, B, C acting on three coordinates. The prover first computes a corank-one point $\hat{a} \in \mathbb{P}(\mathbb{F}_q)$ such that the induced bilinear form $\phi_0(a, y, z)$ is of rank $n - 1$. We run the algorithm

¹⁰ Note that here we introduce a canonical form to reduce the signature size.

$\text{Corank1Cal}(\phi_0)$ to sample a such point. Then we compute the canonical form $\psi = \text{CF}(\phi_0, \hat{a})$. The trilinear form ψ is sent to the verifier as the commitment.

After receiving the commitment, the verifier samples a random challenge $\text{cha} \xleftarrow{\$} \{0, 1\}$. If $\text{cha} = 0$, the prover returns $\text{rsp} = \hat{a}$, and the verifier checks whether $\psi = \text{CF}(\phi_0, \text{rsp})$. If $\text{cha} = 1$, the prover returns $\text{rsp} = \widehat{A^{-1}a}$, and the verifier checks whether $\psi = \text{CF}(\phi_1, \text{rsp})$. The correctness follows from the invariance of the canonical form under the equivalence between ϕ_0 and ϕ_1 , namely $\text{CF}(\phi_0, \hat{a}) = \text{CF}(\phi_1, \widehat{A^{-1}a})$.

1.2 Advantages and limitations

Short invariant responses. The main advantage of TRINE is that a response is a vector rather than a full invertible matrix or a pair of matrices. In the non-base challenge positions, the response size is therefore approximately $n \lceil \log_2 q \rceil$ bits. This is made possible by using the corank-one canonical form as a commitment and checking equality of canonical forms during verification.

Standard GMW-FS structure. TRINE follows the standard GMW identification paradigm and then applies the Fiat-Shamir transform. The proof strategy is therefore the usual one for Fiat-Shamir signatures from special-sound and honest-verifier zero-knowledge identification protocols. The only additional ingredient is the equivariance and extractability of the corank-one canonical form.

Flexible public-key/signature trade-off. As in MEDS and ALTEQ, TRINE supports multiple public forms and fixed-weight challenge strings. Increasing the number X of non-base trilinear forms reduces the number of Fiat-Shamir rounds required for a target security level, and therefore reduces signature size and signing/verification time. The trade-off is a larger public key. The two modes in Section 4 illustrate this trade-off: the *Balanced* mode keeps the public key smaller, while the *ShortSig* mode uses more public forms to reduce signatures and runtime.

Limitations. The main limitation of TRINE is the cost of computing the corank-one canonical form. Each canonical-form computation performs a tripartite walk of length about $3n$, constructs three bases, and applies the resulting transformations to the full trilinear form. The cost per round is therefore roughly $O(n^4)$, which is significantly higher than the $O(n^3)$ -type operations appearing in simpler GMW constructions. This affects signing and verification time, especially for the highest security level.

2 Basic operations

2.1 Basic notations

We use the following notation.

1. $\mathbb{F}_q$, where q is prime: the finite field of order q .
2. $\mathbb{F}_q^n$: the vector space of length- n column vectors.
3. $\mathbb{F}_q^{n \times n}$: the space of $n \times n$ matrices.
4. $\text{GL}(n, q)$: the group of invertible $n \times n$ matrices.
5. $\text{TF}(n, q)$: the vector space of trilinear forms $\mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$.
6. $\mathbb{P}(\mathbb{F}_q^n)$: projective space; $\hat{u}$ denotes the line generated by nonzero u .
7. $[m] = \{1, \dots, m\}$.
8. $x \xleftarrow{\$} S$: uniform random sampling from the finite set S .
9. I_n : the $n \times n$ identity matrix.

2.2 Trilinear forms and a natural group action on them

A trilinear form on $\mathbb{F}_q^n$ is a map $\phi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ that is $\mathbb{F}_q$ -linear in each argument. Denote by $\text{TF}(n, q)$ the linear space of all trilinear forms on $\mathbb{F}_q^n$. Also, a trilinear form $\phi \in \text{TF}(n, q)$ can be represented by coefficients $t_{i,j,k} \in \mathbb{F}_q$ such that $\phi(x, y, z) = \sum_{i,j,k=1}^n t_{i,j,k} x_i y_j z_k$.

The general linear group $\text{GL}(n, q)$ naturally acts on $\text{TF}(n, q)$ as follows: $(A, B, C) \in \text{GL}(n, q)^3$ sends ϕ to $\phi^{A,B,C}$, defined as $\phi^{A,B,C}(x, y, z) = \phi(Ax, By, Cz)$ for all $x, y, z \in \mathbb{F}_q^n$. This action defines an equivalence relation

2.3 TFE, matrix code equivalence and tensor isomorphism

Definition 1 (TFE). *Given two trilinear forms $\phi, \psi : \mathbb{F}_q^m \times \mathbb{F}_q^n \times \mathbb{F}_q^l \rightarrow \mathbb{F}_q$, the problem asks whether there exist three matrices $(A, B, C) \in \text{GL}(m, q) \times \text{GL}(n, q) \times \text{GL}(l, q)$, such that for any $(u, v, w) \in \mathbb{F}_q^m \times \mathbb{F}_q^n \times \mathbb{F}_q^l$, $\phi(u, v, w) = \psi(A(u), B(v), C(w))$.*

Definition 2 (MCE). *Given two matrix codes $\mathcal{C}$ and $\mathcal{D}$ in $\mathbb{F}_q^{m \times n}$, the problem asks whether there exist two invertible matrices $A \in \text{GL}(m, q)$ and $B \in \text{GL}(n, q)$ such that $\mathcal{D} = ACB := \{ACB \mid C \in \mathcal{C}\}$.*

A $[m \times n, l]$ -matrix code $\mathcal{C}$ is an l -dimensional subspace of $\mathbb{F}_q^{m \times n}$. We defined matrix code equivalence in Definition 2. Matrix code equivalence reduces to trilinear form equivalence in polynomial time. This is because of the following. Let a matrix code $\mathcal{C}$ be given by an ordered linear basis $(C_1, C_2, \dots, C_l)$, $C_k \in \mathbb{F}_q^{m \times n}$, and $c_{i,j,k}$ denotes the (i, j) -entry of C_k . This gives rise to a trilinear form $\phi_{\mathcal{C}} : \mathbb{F}_q^m \times \mathbb{F}_q^n \times \mathbb{F}_q^l \rightarrow \mathbb{F}_q$, that is, $\phi_{\mathcal{C}} = \sum_{i,j,k} c_{i,j,k} u_i v_j w_k$ where $u = (u_1, \dots, u_m)^t \in \mathbb{F}_q^m$, $v = (v_1, \dots, v_n)^t \in \mathbb{F}_q^n$, and $w = (w_1, \dots, w_l)^t \in \mathbb{F}_q^l$. It is straightforward to verify that two matrix codes $\mathcal{C}$ and $\mathcal{D}$ are equivalent if and only if $\phi_{\mathcal{C}}$ and $\phi_{\mathcal{D}}$ are equivalent. Furthermore, if $(A, B, C) \in \text{GL}(m, q) \times \text{GL}(n, q) \times \text{GL}(l, q)$ sends $\phi_{\mathcal{C}}$ to $\phi_{\mathcal{D}}$, then (A, B) sends $\mathcal{C}$ to $\mathcal{D}$.

2.4 Corank-one points and the canonical form

Instantiated arguments of trilinear forms. Let $\phi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ be a trilinear form and $u, v \in \mathbb{F}_q^n$. We use $\phi(u, \star, \star)$ to denote the bilinear form obtained by instantiating the first argument of ϕ with u . Let $\phi(u, \star, \star) = \sum_{j,k} c_{j,k} y_j z_k$, then it has matrix representation $M_u = (c_{j,k})$ with respect to standard basis $e_1, \dots, e_n$. We use $\phi(u, v, \star)$ to denote the linear form obtained by instantiating the first two arguments of ϕ with u and v , respectively.

Tripartite graphs associated with trilinear forms. Let $\phi \in \text{TF}(n, q)$ be a trilinear form, then we can associate ϕ with a tripartite graph $G_{\phi} = (U \sqcup V \sqcup W, E)$ where $U = V = W = \mathbb{P}(\mathbb{F}_q^n)$. To define the edge set E , let $\hat{u} \in U$, $\hat{v} \in V$, and $\hat{w} \in W$. Then $\{\hat{u}, \hat{v}\} \in E$, if $\phi(u, v, \star)$ is the zero linear form. Similarly, $\{\hat{u}, \hat{w}\} \in E$, if $\phi(u, \star, w)$ is the zero linear form. And $\{\hat{v}, \hat{w}\} \in E$, if $\phi(\star, v, w)$ is the zero linear form.

Given a trilinear form $\phi \in \text{TF}(n, q)$, a corank-one projective point of ϕ is a projective point $\hat{u} \in \mathbb{P}(\mathbb{F}_q^n)$ such that the bilinear form $\phi(u, \star, \star)$ is of rank $n - 1$. In the following, we introduce the canonical form of a trilinear form $\phi \in \text{TF}(n, q)$ with respect to a corank-one projective point $\hat{u}$, denoted by $\text{CF}(\phi, \hat{u})$. If two trilinear forms ϕ and $\psi = \phi^{A,B,C}$ are equivalent, then they have the same canonical form with respect to $\hat{u}$ and $A^{-1}\hat{u}$, respectively. Therefore, this canonical form can be viewed as an invariant for equivalent trilinear forms.

The main idea To make this invariant effective, we need a systematic way to obtain enough corank-one points from a given trilinear form. Recall that a tripartite graph $G_\phi = (U \sqcup V \sqcup W, E)$ is associated with ϕ , where $U = V = W = \mathbb{P}(\mathbb{F}_q^n)$. Starting from a corank-one point $\hat{u} \in U$, the bilinear form $\phi(u, \star, \star)$ has rank $n - 1$, and hence has a unique one-dimensional left kernel. This kernel determines a unique neighbour $\hat{v} \in V$ satisfying that $\phi(u, v, \star)$ is the zero linear form. Since $\phi(\star, v, \star)$ has u in its left kernel, it has corank at least one. If $\phi(\star, v, \star)$ is again of corank one, then it determines a unique neighbour $\hat{w} \in W$. Repeating this procedure gives a path in G_ϕ . We continue this process until the path has length $3n$, collecting n points from each of U , V , and W . The path is constructed without ambiguity exactly when a corank-one point is obtained at every step.

To make this an isomorphism invariant indexed with $\hat{u}$, instead of with u , we need to remove the ambiguity caused by scalar multiples. This can be done easily by locating non-zero evaluations of $\tilde{\phi}|_u$ on about $3n$ inputs of the form (e_i, e_j, e_k) . This gives us $\tilde{\phi}[\hat{u}]$, which is an invariant associated with $\hat{u}$.

We now describe how to construct $\text{CF}(\phi, \hat{u})$ from a corank-one projective point $\hat{u}$.

Constructing the cononical form Given an arbitrary corank-one point $\hat{u}$. Since $\phi(u_1, \star, \star)$ has corank one, there is a unique projective point $\hat{v}_1 \in \mathbb{P}(\mathbb{F}_q^n)$ such that $\phi(u_1, v_1, \star)$ is the zero linear form. If $\phi(\star, v_1, \star)$ also has corank one, then there is a unique projective point $\hat{w}_1 \in \mathbb{P}(\mathbb{F}_q^n)$ such that $\phi(\star, v_1, w_1)$ is the zero linear form. Similarly, if $\phi(\star, \star, w_1)$ has corank one, then there is a unique projective point $\hat{u}_2 \in \mathbb{P}(\mathbb{F}_q^n)$ such that $\phi(u_2, \star, w_1)$ is the zero linear form.

Repeating this procedure, we obtain three sequences of projective points $\hat{u}_1, \dots, \hat{u}_n$, $\hat{v}_1, \dots, \hat{v}_n$, and $\hat{w}_1, \dots, \hat{w}_n$. More precisely, after $\hat{u}_i$ has been obtained, we compute $\hat{v}_i$ from the left kernel of $\phi(u_i, \star, \star)$, then compute $\hat{w}_i$ from the kernel condition $\phi(\star, v_i, w_i) = 0$, and finally compute $\hat{u}_{i+1}$ from the kernel condition $\phi(u_{i+1}, \star, w_i) = 0$. If at some step the corresponding bilinear form does not have corank one, or if the newly obtained point is linearly dependent on the previously collected points in the same part, then the construction fails for this starting point. Otherwise, after n successful rounds, we obtain three linearly independent tuples $L_U = (u_1, \dots, u_n)$, $L_V = (v_1, \dots, v_n)$, and $L_W = (w_1, \dots, w_n)$.

We regard L_U , L_V , and L_W as invertible matrices whose columns are the corresponding vectors. They define a transformed trilinear form $\tilde{\phi}_{\hat{u}}$ by

$$\tilde{\phi}_{\hat{u}}(x, y, z) = \phi(L_U x, L_V y, L_W z).$$

This form is determined by the path starting from $\hat{u}$, except for the choice of representatives of the projective points $\hat{u}_i$, $\hat{v}_i$, and $\hat{w}_i$. Changing these representatives only acts on $\tilde{\phi}_{\hat{u}}$ by invertible diagonal matrices in the three coordinates. Therefore, to obtain an invariant depending only on $\hat{u}$, we normalize this diagonal ambiguity.

Let $F = \text{diag}(f_1, \dots, f_n)$, $G = \text{diag}(g_1, \dots, g_n)$, and $H = \text{diag}(h_1, \dots, h_n)$ be invertible diagonal matrices. Assume that the following entries of $\tilde{\phi}_{\hat{u}}$ are non-zero:

$$a_i = \tilde{\phi}_{\hat{u}}(e_i, e_2, e_1), \quad b_j = \tilde{\phi}_{\hat{u}}(e_1, e_j, e_1), \quad c_k = \tilde{\phi}_{\hat{u}}(e_1, e_2, e_k)$$

for $3 \leq i, j, k \leq n$, together with

$$d_1 = \tilde{\phi}_{\hat{u}}(e_1, e_2, e_1), \quad d_2 = \tilde{\phi}_{\hat{u}}(e_2, e_3, e_5), \quad d_3 = \tilde{\phi}_{\hat{u}}(e_1, e_3, e_2), \quad d_4 = \tilde{\phi}_{\hat{u}}(e_2, e_1, e_2).$$

We choose F , G , and H so that all these entries become equal to 1 after the diagonal transformation. Equivalently, we impose

$$f_1 g_2 h_1 = d_1^{-1}, \quad \frac{f_i}{f_1} = \frac{d_1}{a_i}, \quad \frac{g_j}{g_2} = \frac{d_1}{b_j}, \quad \frac{h_k}{h_1} = \frac{d_1}{c_k}$$

for $3 \leq i, j, k \leq n$, and additionally

$$f_2 = (g_3 h_5 d_2)^{-1}, \quad h_2 = (f_1 g_3 d_3)^{-1}, \quad g_1 = (f_2 h_2 d_4)^{-1}.$$

Then we define

$$\bar{\phi}_{\hat{u}}(x, y, z) = \tilde{\phi}_{\hat{u}}(Fx, Gy, Hz).$$

By construction, the selected entries of $\bar{\phi}_{\hat{u}}$ are all normalized to 1. This removes the ambiguity caused by scalar representatives of the projective points. We define the canonical form of ϕ with respect to $\hat{u}$ as

$$\mathbf{CF}(\phi, \hat{u}) := \bar{\phi}_{\hat{u}}.$$

If the path construction succeeds and the required entries are non-zero, then $\mathbf{CF}(\phi, \hat{u})$ is well-defined. Moreover, this construction is canonical with respect to the projective starting point $\hat{u}$, and hence gives an invariant associated with $\hat{u}$.

Matrix Description Let $\phi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ be a trilinear form. In the following, we represent ϕ by a sequence of matrices $\mathcal{M} = (M_1, \dots, M_n)$, $M_k \in \mathbb{F}_q^{n \times n}$, such that for any $x, y, z \in \mathbb{F}_q^n$, $\phi(x, y, z) = x^\top (\sum_{k=1}^n z_k M_k) y$. For $z = (z_1, \dots, z_n)^\top$, define the matrix $M(z) := \sum_{k=1}^n z_k M_k$. Thus, fixing the third argument z , the trilinear form ϕ induces the bilinear form $(x, y) \mapsto x^\top M(z)y$.

We also define the matrices obtained by fixing the first or the second argument of ϕ . For $u \in \mathbb{F}_q^n$, let

$$\Phi_U(u) := [M_1^\top u, M_2^\top u, \dots, M_n^\top u].$$

Then for any $y, z \in \mathbb{F}_q^n$, $\phi(u, y, z) = y^\top \Phi_U(u)z$.

Similarly, for $v \in \mathbb{F}_q^n$, let

$$\Phi_V(v) := [M_1 v, M_2 v, \dots, M_n v].$$

Then for any $x, z \in \mathbb{F}_q^n$, $\phi(x, v, z) = x^\top \Phi_V(v)z$.

Therefore,

$$\phi(u, v, w) = v^\top \Phi_U(u)w = u^\top \Phi_V(v)w = u^\top M(w)v.$$

In this sense, a projective point $\hat{u} \in \mathbb{P}(\mathbb{F}_q^n)$ is called a corank-one point of ϕ if $\text{rank}(\Phi_U(u)) = n - 1$. This condition is independent of the choice of the representative u , since replacing u by a nonzero scalar multiple only scales the matrix $\Phi_U(u)$.

In the algorithms below, we use the following elementary linear-algebra subroutines. For a matrix A , $\text{Rank}(A)$ denotes its rank. If A has a one-dimensional left kernel, then $\text{LKer}(A)$ returns a nonzero vector x satisfying $x^\top A = 0$. If A has a one-dimensional right kernel, then $\text{RKer}(A)$ returns a nonzero vector x satisfying $Ax = 0$. For a vector x and a list of vectors L , the procedure $\text{InSpan}(x, L)$ returns true if and only if $x \in \text{span}(L)$. All vectors returned by LKer and RKer are only determined up to nonzero scalar multiples. This scalar ambiguity will be removed later by a diagonal normalization step.

Algorithm 1: Corank1Cal(ϕ)

Input: A trilinear form $\phi \in \text{TF}(n, q)$, represented by matrices

$$M_1, \dots, M_n \in \mathbb{F}_q^{n \times n}.$$

Output: A corank-1 point $u \in \mathbb{F}_q^n$.

```

1 repeat
2   |  $u \xleftarrow{\$} \mathbb{F}_q^n$ ;
3   |  $r = \text{Rank}(\Phi_U(u))$ ;
4 until  $r = n - 1$ ;
5 return  $u$ ;
```

Algorithm 2: BuildUVW($\mathcal{M}, u_1$)

Input: A matrix representation $\mathcal{M} = (M_1, \dots, M_n)$ of a trilinear form ϕ , and a corank-1 point $u_1 \in \mathbb{F}_q^n$.

Output: Either $\perp$, or three matrices $U, V, W \in \text{GL}(n, q)$.

```

1 if Rank( $\Phi_U(u_1)$ )  $\neq n - 1$  then
2   return  $\perp$ ;
3  $L_U \leftarrow (u_1)$ ,  $L_V \leftarrow ()$ ,  $L_W \leftarrow ()$ ;
4 for  $i = 1$  to  $n$  do
5    $v_i \leftarrow \text{LKer}(\Phi_U(u_i))$ ;
6   if  $\text{InSpan}(v_i, L_V)$  then
7     return  $\perp$ ;
8   Append  $v_i$  to  $L_V$ ;
9   if Rank( $\Phi_V(v_i)$ )  $\neq n - 1$  then
10    return  $\perp$ ;
11    $w_i \leftarrow \text{RKer}(\Phi_V(v_i))$ ;
12   if  $\text{InSpan}(w_i, L_W)$  then
13     return  $\perp$ ;
14   Append  $w_i$  to  $L_W$ ;
15   if  $i = n$  then
16     break;
17   if Rank( $M(w_i)$ )  $\neq n - 1$  then
18     return  $\perp$ ;
19    $u_{i+1} \leftarrow \text{LKer}(M(w_i))$ ;
20   if Rank( $\Phi_U(u_{i+1})$ )  $\neq n - 1$  then
21     return  $\perp$ ;
22   if  $\text{InSpan}(u_{i+1}, L_U)$  then
23     return  $\perp$ ;
24   Append  $u_{i+1}$  to  $L_U$ ;
25 Set  $U = [u_1, u_2, \dots, u_n]$ ,  $V = [v_1, v_2, \dots, v_n]$ ,  $W = [w_1, w_2, \dots, w_n]$ ;
26 if  $\det(U) \det(V) \det(W) = 0$  then
27   return  $\perp$ ;
28 return  $(U, V, W)$ ;

```

Algorithm 3: DiagonalNormalize($\mathcal{M}, U, V, W$)

Input: A matrix representation $\mathcal{M} = (M_1, \dots, M_n)$ of ϕ , and three matrices $U, V, W \in \text{GL}(n, q)$.

Output: Either $\perp$, or a normalized matrix representation $\overline{\mathcal{M}} = (\overline{M}_1, \dots, \overline{M}_n)$

```

1 Write  $W = [w_1 \ w_2 \ \dots \ w_n]$ ;
2 for  $k = 1$  to  $n$  do
3    $\widetilde{M}_k \leftarrow U^\top M(w_k) V$  .
4 for  $i, j, k = 3$  to  $n$  do
5    $a_i \leftarrow (\widetilde{M}_1)_{i,2}, b_j \leftarrow (\widetilde{M}_1)_{1,j}, c_k \leftarrow (\widetilde{M}_k)_{1,2}$ ;
6  $d_1 \leftarrow (\widetilde{M}_1)_{1,2}, d_2 \leftarrow (\widetilde{M}_5)_{2,3}, d_3 \leftarrow (\widetilde{M}_2)_{1,3}, d_4 \leftarrow (\widetilde{M}_2)_{2,1}$  ;
7 if some element among  $a_i, b_j, c_k, d_1, d_2, d_3, d_4$  is zero then
8   return  $\perp$ ;
9 Set  $f_1 \leftarrow 1, g_2 \leftarrow 1, h_1 \leftarrow d_1^{-1}$  ;
10 for  $i = 3$  to  $n$  do
11    $f_i \leftarrow \frac{d_1}{a_i}$ 
12 for  $j = 3$  to  $n$  do
13    $g_j \leftarrow \frac{d_1}{b_j}$ 
14 for  $k = 3$  to  $n$  do
15    $h_k \leftarrow \frac{h_1 d_1}{c_k}$ 
16 Set  $f_2 \leftarrow (g_3 h_5 d_2)^{-1}, h_2 \leftarrow (f_1 g_3 d_3)^{-1}, g_1 \leftarrow (f_2 h_2 d_4)^{-1}$ ;
17 Set  $F \leftarrow \text{diag}(f_1, \dots, f_n), G \leftarrow \text{diag}(g_1, \dots, g_n), H \leftarrow \text{diag}(h_1, \dots, h_n)$ ;
18 for  $k = 1$  to  $n$  do
19    $\overline{M}_k \leftarrow h_k F \widetilde{M}_k G$ 
20 return  $(\overline{M}_1, \dots, \overline{M}_n)$ ;
```

Algorithm 4: CF($\mathcal{M}, u$)

Input: A matrix representation $\mathcal{M} = (M_1, \dots, M_n)$ of ϕ , and a corank-one point $u \in \mathbb{F}_q^n$.

Output: Either $\perp$, or $\text{CF}_\phi(u) = (\overline{M}_1, \dots, \overline{M}_n)$.

```

1  $(U, V, W) \leftarrow \text{BuildUVW}(\mathcal{M}, u)$ ;
2 if  $(U, V, W) = \perp$  then
3   return  $\perp$ ;
4  $\overline{\mathcal{M}} \leftarrow \text{DiagonalNormalize}(\mathcal{M}, U, V, W)$ ;
5 if  $\overline{\mathcal{M}} = \perp$  then
6   return  $\perp$ ;
7  $\text{CF}_\phi(\hat{u}_1) \leftarrow \overline{\mathcal{M}}$ ;
8 return  $\text{CF}_\phi(u)$ ;
```

3 The TRINE signature scheme

3.1 Parameters

A TRINE parameter set specifies the following values.

1. n : the vector-space dimension in all three tensor modes.
2. q : the prime field order.
3. λ : the target classical security level in bits.
4. r : the number of parallel identification rounds.
5. $X + 1$: the number of public trilinear forms and challenge values.
6. K : the fixed number of non-base challenges.
7. M : the message.

3.2 Key generation

The key generation procedure is described in Algorithm 5. Here we summarise the formats of private and public keys.

Private key. The private key consists of $3X$ invertible matrices $A_0, \dots, A_{X-1}, B_0, \dots, B_{X-1}, C_0, \dots, C_{X-1} \in \text{GL}(n, q)$. The private key consists of

$$3X \cdot n^2$$

field elements. Note that the private key can be generated by a pseudo-random generator, so we only need a random seed as a private key.

Public Key. The public key consists $X + 1$ trilinear forms $\phi_0, \dots, \phi_X \in \text{TF}(n, q)$ such that $\phi_X^{A_i, B_i, C_i} = \phi_i$ for $i \in \{0, \dots, X - 1\}$. The public key consists of

$$(X + 1) \cdot n^3$$

field elements.

Algorithm 5: Key generation

Input: The variable number $n \in \mathbb{N}$, a prime power q , the trilinear form number $X + 1$

Output: Public key: $X + 1$ trilinear forms $\phi_i \in \text{TF}(n, q)$ such that $\phi_i \cong \phi_j$ for any $i, j \in \{0, \dots, X\}$

Private key: $3X$ matrices $\{(A_0, B_0, C_0), \dots, (A_{X-1}, B_{X-1}, C_{X-1})\}$ such that $\phi_i = \phi_X^{A_i, B_i, C_i}$

- 1 Randomly sample a trilinear form $\phi_X : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$;
 - 2 Randomly sample $3X$ invertible matrices $(A_0, \dots, A_{X-1}, B_0, \dots, B_{X-1}, C_0, \dots, C_{X-1}) \in \text{GL}(n, q)$
 - 3 For every $i \in \{0, \dots, X - 1\}$, $\phi_i \leftarrow \phi_X^{A_i, B_i, C_i}$
 - 4 **return** Public Key: $\phi_0, \dots, \phi_X$. Private key: $\{(A_0, B_0, C_0), \dots, (A_{X-1}, B_{X-1}, C_{X-1})\}$.
-

3.3 Signature generation and verification

The algorithms 6 and 7 describe the signature generation and verification processes of our basic scheme.

Algorithm 6: Signature generation process.

Input: The public key $\phi_0, \dots, \phi_X \in \text{TF}(n, q)$. The Private key $(A_0, \dots, A_{X-1}, B_0, \dots, B_{X-1}, C_0, \dots, C_{X-1}) \in \text{GL}(n, q)$. $r, X, \lambda \in \mathbb{N}$. Let $A_X = B_X = C_X = I$, the identity matrix. The message M . A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$.

Output: The signature S on M .

```

1 for  $i \in \{0, \dots, r-1\}$  do
2   repeat
3      $a_i \leftarrow \text{Corank1Cal}(\phi_X)$ ;
4      $\psi_i \leftarrow \text{CF}(\phi_X, a_i)$ ;
5   until  $\psi_i \neq \perp$ ;
6 Compute  $\text{cha} = H(M || \psi_0 || \dots || \psi_{r-1}) \in \{0, 1\}^{2\lambda}$ 
7  $(b_0, \dots, b_{r-1}) \leftarrow \text{ParseHash}(\text{cha})$ 
8 for  $i \in \{0, \dots, r-1\}$  do
9    $d_i \leftarrow A_{b_i}^{-1} a_i$ 
10 return  $S = (\text{cha}, d_0, \dots, d_{r-1})$ .
```

Algorithm 7: Verification procedure.

Input: The public key $\phi_0, \dots, \phi_X \in \text{TF}(n, q)$. The signature $S = (\text{cha}, d_0, \dots, d_{r-1})$, $d_i \in \mathbb{F}_q^n$. The message M . A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$.

Output: “Yes” if S is a valid signature for M . “No” otherwise.

```

1 for  $i \in \{0, \dots, r-1\}$  do
2    $\psi'_i \leftarrow \text{CF}(\phi_{b_i}, d_i)$ 
3 Compute  $\text{cha}' = H(M || \psi'_0 || \dots || \psi'_{r-1}) \in \{0, 1\}^{2\lambda}$ 
4  $(b'_0, \dots, b'_{r-1}) \leftarrow \text{ParseHash}(\text{cha}')$ 
5  $(b_0, \dots, b_{r-1}) \leftarrow \text{ParseHash}(\text{cha})$ 
6 if for every  $i \in \{0, \dots, r-1\}, b_i = b'_i$  then
7   return Yes
8 else
9   return No
```

3.4 Unbalanced challenge space

In Algorithm 6, note that when $b_i = X$, we have $d_i = a_i$ that is not related to the private key. Therefore, we can use a seed in place of a_i , which saves signature size as we only need to send a seed of size λ , rather than n field elements. This observation leads to unbalancing the challenge space (as discussed already in [FS86]), reducing the signature size. Specifically, we will sample r challenges $(b_0, \dots, b_{r-1}) \in \{0, \dots, X\}^r$ with the property that $|\{i \in [r] \mid b_i = X\}| = r - K$. To achieve λ bits of security, we need to choose proper parameters r, K and X , such that $\binom{r}{K} \cdot X^K \geq 2^\lambda$.

In the signature generation process, we use an algorithm **ParseHash** (Algorithm 8) to implement the above procedure as follows.

Algorithm 8: ParseHash $_{X,r,K}(\mathcal{B}^{\ell_{\text{digest}}})$

Input: digest $d \in \mathcal{B}^{\ell_{\text{digest}}}$ as byte string, parameters $X \in \{2, \dots, 256\}$, $r, K \in \mathbb{Z}^+$,
 $w \leq r$

Output: $b_0, \dots, b_{r-1} \in \{0, \dots, X\}$ such that K elements are not X .

```

1  $x_0, x_1, \dots \in \mathcal{B}^* \leftarrow \text{XOF}(d, *)$ ;
2  $\rho \leftarrow 0$ ;
3 forall  $i \in \{0, \dots, r-1\}$  do
4    $b_i \leftarrow 0$ ;
5 forall  $i \in \{0, \dots, K-1\}$  do
6    $\alpha \leftarrow 0$ ;
7   forall  $j \in \{0, \dots, \text{ByteLen}(r)-1\}$  do
8      $\alpha \leftarrow \alpha + x_\rho 2^{8j}$ ;
9      $\rho \leftarrow \rho + 1$ ;
10   $\alpha \leftarrow \alpha \bmod 2^{\text{BitLen}(r)}$ ;
11  if  $\alpha \geq r$  or  $b_\alpha > 0$  then
12    goto line 5;
13   $b_\alpha \leftarrow x_\rho$ ;
14   $\rho \leftarrow \rho + 1$ ;
15   $b_\alpha \leftarrow b_\alpha \bmod 2^{\text{BitLen}(X+1)}$ ;
16  if  $b_\alpha = 0$  or  $b_\alpha \geq X+1$  then
17    goto line 12;
18 return  $X - b_0, \dots, X - b_{r-1} \in \{0, \dots, X\}$ ;
```

3.5 The formulas for public key, private key, and signature sizes

The following formulas give the public key, private key, and signature sizes in *bits*, based on the parameters n, q, r, K and X .

$$\begin{aligned}
\text{PubKeySize} &= X \cdot n^3 \cdot \lceil \log q \rceil + 2\lambda, \\
\text{PriKeySize} &= 2\lambda, \\
\text{SigSize} &= (r - K + 2) \cdot \lambda + K \cdot n \cdot \lceil \log q \rceil.
\end{aligned}$$

It should be noted that the private key consists of a seed of length 2λ which generates the $3X$ private matrices. The public key consists of X trilinear forms and a seed of length 2λ which generates ϕ_X . The signature consists of K corank-1 points, $r - K$ seeds of length λ and a challenge of length 2λ .

4 Performance analysis

We provide two sets of parameters for each bit security 128, 256 and 512. The first set is called *Balanced*, and the second set is called *ShortSig*.

4.1 Key and signature sizes

Table 1: Key and Signature Sizes for Balanced-TRINE

Parameter set	Parameters (n, q, r, K, X)	Private key Size (Bytes)	Public key Size (Bytes)	Signature Size (Bytes)
TRINE-128	(22, 4093, 138, 52, 1)	32	16004	3124
TRINE-256	(40, 4093, 271, 104, 1)	64	96064	11648
TRINE-512	(81, 4093, 534, 211, 1)	128	797290	46437

Table 2: Key and Signature Sizes for ShortSig-TRINE

Parameter set	Parameters (n, q, r, K, X)	Private key Size (Bytes)	Public key Size (Bytes)	Signature Size (Bytes)
TRINE-128	(22, 4093, 61, 36, 4)	32	63920	1620
TRINE-256	(40, 4093, 116, 76, 4)	64	384064	5904
TRINE-512	(81, 4093, 232, 149, 4)	128	3188774	23544

4.2 Performance

We evaluate the performance of TRINE on the following platform: AMD Ryzen AI9 HX370 (2.0 GHz Base, 5.1 GHz Turbo) Processor, x86 architecture. The RAM is 32 GiB. The software environment is WSL Ubuntu 24.04.4 LTS, gcc 11.4.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

Our results are list in Table 3 and Table 4 as follows. The numbers in the following Tables are averages over 100 runs. We report the averages, and the medians are quite close to the averages.

Table 3: Performance of Balanced-TRINE (cycles).

Parameter set	KeyGen	Sign	Verify
TRINE-128	1547804	10345068140	5840471423
TRINE-256	10172744	114493979025	72536805882
TRINE-512	124758990	1977890653443	1258486639726

Table 4: Performance of ShortSig-TRINE.

parameter set	KeyGen	Sign	Verify
TRINE-128	4868274	4875380743	2167962778
TRINE-256	36281018	50839302379	19188586288
TRINE-512	470938518	897905055483	389535751708

5 Security reductions

5.1 Assumptions

The security of our scheme **TRINE** is based on the following several assumptions.

Problem 1 (psTFE). The promised search version of the trilinear form equivalence problem (**psTFE**) is the following. Given two trilinear forms $\phi, \psi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$, with the promise that $\phi \sim \psi$, output $A, B, C \in \text{GL}(n, q)^3$ such that $\psi = \phi^{A,B,C}$.

Problem 2 (ℓ -psTFE). The promised search version of the trilinear form equivalence problem with ℓ -instances (ℓ -**psTFE**) is the following. Given ℓ trilinear forms $\phi_1, \dots, \phi_\ell : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$, with the promise that $\phi_i \sim \phi_j$ for any $i, j \in [\ell]$, output some $(A, B, C) \in \text{GL}(n, q)^3$ and $i, j \in [\ell]$, $i \neq j$, such that $\phi_i = \phi_j^{A,B,C}$.

Problem 3 (ℓ -PR-psTFE-RO). The pseudorandom trilinear form equivalence problem with ℓ random instances (ℓ -**PR-psTFE-RO**) asks to distinguish the following two distributions.

The random distribution: ℓ trilinear forms $\phi_0, \dots, \phi_{\ell-1} : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$, drawn as $(\phi_0, \dots, \phi_{\ell-1}) \xleftarrow{\$} \text{TF}(n, q)^\ell$.

The pseudorandom distribution: ℓ trilinear forms $\phi_0, \dots, \phi_{\ell-1} : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ such that: $\phi_0 \xleftarrow{\$} \text{TF}(n, q)$, and for each $i \in [\ell - 1]$, $\phi_i = \phi_0^{A_i, B_i, C_i}$, where $(A_i, B_i, C_i) \in \text{GL}(n, q)^3$.

Problem 4 (CF-TFE). The canonical form trilinear form equivalence problem (**CF-TFE**) is the following. Given two trilinear forms $\phi, \psi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$, output two projective points $\hat{u}, \hat{v} \in \mathbb{P}(\mathbb{F}_q^n)$, such that $\text{CF}(\phi, \hat{u}) = \text{CF}(\psi, \hat{v})$.

We give a brief reduction between **psTFE** problem and **CF-TFE** problem. On one hand, given a solution to **psTFE** problem, i.e., the isomorphism matrices (A, B, C) such that $\psi = \phi^{A,B,C}$. Then any valid corank-one point $\hat{u}$ of ϕ implies a valid corank-one point $\hat{v} = \widehat{A^{-1}u}$ such that $\text{CF}(\phi, \hat{u}) = \text{CF}(\psi, \hat{v})$. On the other hand, if one has a pair of projective point $\hat{u}, \hat{v} \in \mathbb{P}(\mathbb{F}_q^n)$ such that $\text{CF}(\phi, \hat{u}) = \text{CF}(\psi, \hat{v})$. Then the isomorphism matrices between ϕ and ψ can be obtained by the paths from ϕ and ψ to the same canonical form.

5.2 Security of **TRINE**

The EUF-CMA security proof of **TRINE** follows the standard methodology for signatures obtained by applying the Fiat–Shamir transformation to a **GMW** identification protocol arising from a group action. The main difference from the basic **GMW** construction is that a **TRINE** response is not a complete group element. Instead, it is an effective corank-one projective point whose canonical form serves as the commitment. For clarity, we first establish completeness, 2-special soundness, and honest-verifier zero knowledge for the basic one-round binary protocol of Figure 1, whose challenge space is $\{0, 1\}$. The extension to full **TRINE** protocol is discussed at the end of this subsection.

Completeness. Straightforward to check.

2-Special Soundness. Given two accepting transcripts with the same commitment and distinct challenges, an extractor can compute a solution for the underlying TFE problem. Let $(\psi, 0, \text{rsp}_0)$ and $(\psi, 1, \text{rsp}_1)$ be two valid transcripts. Since both pass verification, we have that $\text{CF}(\phi_0, \text{rsp}_0) = \text{CF}(\phi_1, \text{rsp}_1)$. Thanks to the canonical form computation procedure, we can compute $(U_0, V_0, W_0) \leftarrow \text{BuildUVW}(\phi_0, \text{rsp}_0)$ and $(U_1, V_1, W_1) \leftarrow \text{BuildUVW}(\phi_1, \text{rsp}_1)$. Further, we can compute the diagonal matrices (F_0, G_0, H_0) and (F_1, G_1, H_1) from $\text{DiagonalNormalize}(\phi_0, U_0, V_0, W_0)$ and $\text{DiagonalNormalize}(\phi_1, U_1, V_1, W_1)$. It is easy to see that $\phi_0^{U_0 F_0, V_0 G_0, W_0 H_0} = \phi_1^{U_1 F_1, V_1 G_1, W_1 H_1} = \text{CF}(\phi_0, \text{rsp}_0) = \text{CF}(\phi_1, \text{rsp}_1)$. Thus we have $\phi_0^{U_0 F_0 F_1^{-1} U_1^{-1}, V_0 G_0 G_1^{-1} V_1^{-1}, W_0 H_0 H_1^{-1} W_1^{-1}} = \phi_1$, i.e. $(U_0 F_0 F_1^{-1} U_1^{-1}, V_0 G_0 G_1^{-1} V_1^{-1}, W_0 H_0 H_1^{-1} W_1^{-1})$ is a solution to the TFE problem.

Honest-Verifier Zero Knowledge. To show this, we provide a simulator S which, without the knowledge of the witness, is able to produce a transcript which is indistinguishable from one obtained after an interaction with an honest verifier.

The simulator S receives the public key and a prescribed challenge $\text{cha} \in \{0, 1\}$. It performs the following steps:

1. sample $\hat{u} \leftarrow \text{Corank1Cal}(\phi_{\text{cha}})$;
2. compute $\psi = \text{CF}(\phi_c, \hat{u})$;
3. output the transcript $(\psi, \text{cha}, \hat{u})$.

We compare this transcript with a real transcript conditioned on challenge cha . In a real execution $\hat{a} \leftarrow \text{Corank1Cal}(\phi_0)$ and the response is $\hat{a}$ if $\text{cha} = 0$ otherwise $A^{-1}\hat{a}$. When $\hat{a}$ is uniformly generated, then the vector $A^{-1}\hat{a}$ is uniformly generated too, and hence follow the same distribution as $\hat{u}$. Therefore the simulated and real responses have identical distributions.

Extension to the full TRINE protocol. The arguments above were stated for the basic binary protocol of Figure 1. The complete TRINE scheme contains $X + 1$ equivalent public forms $\phi_0, \dots, \phi_X$ and uses challenges from $\{0, \dots, X\}$.

Completeness and honest-verifier zero knowledge extend by replacing (ϕ_0, ϕ_1) with the challenged form ϕ_c . For special soundness, two accepting transcripts with the same commitment and two distinct challenges $c \neq c'$ yield equal canonical forms for points of ϕ_c and $\phi_{c'}$. Composing the two canonicalization certificates gives a full equivalence between these two public forms.

5.3 Algorithms of the TFE problem

Algebraic Attack The algebraic attack considered in this section is adapted from the algebraic attack against matrix code equivalence described in [CNP⁺23a], which extends the improved modeling of [CNP⁺23b]. The main idea is to view the trilinear form equivalence problem as a structured polynomial system in the entries of the secret transformation matrices. Instead of directly solving a cubic system, one derives several bilinear systems by eliminating inverse matrices through suitable kernel spaces.

Let $\phi, \psi : \mathbb{F}_q^n \times \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ be two equivalent trilinear forms. The TFE problem asks to recover $A, B, C \in \text{GL}(n, q)$ such that $\phi(Ax, By, Cz) = \psi(x, y, z)$ for all $x, y, z \in \mathbb{F}_q^n$. Directly expanding this relation gives cubic equations in the $3n^2$ unknown entries of

A, B, C . To obtain a more structured system, we use the following equivalent identities:

$$\phi(Ax, By, z) = \psi(x, y, C^{-1}z), \quad (1)$$

$$\phi(Ax, y, Cz) = \psi(x, B^{-1}y, z), \quad (2)$$

$$\phi(x, By, Cz) = \psi(A^{-1}x, y, z). \quad (3)$$

We explain how to eliminate C^{-1} from (1); the other two cases are analogous. For $\omega \in \mathbb{F}_q^{n \times n}$ with a decomposition $\omega = \sum_{\alpha} v_{\alpha} w_{\alpha}^{\top}$, define $\psi(\omega, z) = \sum_{\alpha} \psi(v_{\alpha}, w_{\alpha}, z)$. By trilinearity, this definition is independent of the decomposition. Consider the kernel space

$$K_{12}(\psi) = \{\omega \in \mathbb{F}_q^{n \times n} : \psi(\omega, z) = 0, \forall z \in \mathbb{F}_q^n\}.$$

Since this imposes at most n linear constraints on an n^2 -dimensional space, we have $\dim K_{12}(\psi) \geq n^2 - n$.

Let $\{\omega_{\beta}\}_{\beta}$ be a basis of $K_{12}(\psi)$ and write $\omega_{\beta} = \sum_{\alpha} v_{\beta, \alpha} w_{\beta, \alpha}^{\top}$. Evaluating (1) on ω_{β} gives

$$\sum_{\alpha} \phi(Av_{\beta, \alpha}, Bw_{\beta, \alpha}, z) = \sum_{\alpha} \psi(v_{\beta, \alpha}, w_{\beta, \alpha}, C^{-1}z) = 0.$$

Taking $z = e_1, \dots, e_n$ gives n scalar equations for each ω_{β} . Hence the first family yields $n(n^2 - n)$ bilinear equations of tri-degree $(1, 1, 0)$ in the entries of A and B . Similarly, (2) gives $n(n^2 - n)$ equations of tri-degree $(1, 0, 1)$ in A and C , while (3) gives $n(n^2 - n)$ equations of tri-degree $(0, 1, 1)$ in B and C . Altogether, we obtain $3n(n^2 - n)$ quadratic equations in $3n^2$ unknowns.

This approach can also be hybridized, as described in [CNP⁺23a]. Since all three dimensions are n , one may fix two rows of A , namely $2n$ variables, and then derive a linear system for B . This gives a classical complexity of $O(q^{2n}n^6)$. If the enumeration step is accelerated by Grover search, the corresponding quantum complexity becomes $O(q^n n^6)$. However, as in the matrix-code-equivalence setting, this hybrid approach does not necessarily outperform the full algebraic attack for parameter sets over large fields.

Invariant-based attack Two recent invariant-based algorithms are particularly relevant to the security of **TRINE**. The first one, due to Narayanan, Qiao, and Tang [NQT24], associates canonical forms with corank-one points of a trilinear form. The second one, due to Couvreur and Levrat [CL25], represents a trilinear form as a matrix code and uses invariants derived from one-dimensional hulls of related matrix codes.

Although the two algorithms use different invariants, both exploit the same general strategy: construct two lists of invariant values for the two equivalent public trilinear forms, find a collision between these lists, and use the matched objects to recover the hidden equivalence. We account for both attacks in the selection of the **TRINE** parameters.

Corank-one invariant attack. The attack introduced in [NQT24] uses the canonical form induced by corank-one points as a distinguishing invariant for trilinear form equivalence. Since the computation of $\mathbf{CF}(\phi, \hat{u})$ has already been described above, we only explain how it is used to recover the equivalence matrices.

Let $\phi, \psi \in \mathbf{TF}(n, q)$ be two equivalent trilinear forms. Assume that there exist $(A, B, C) \in \mathbf{GL}(n, q)^3$ such that $\phi^{A, B, C} = \psi$. Then the action also induces a correspondence between projective points in the first coordinate. In particular, if $\hat{u}$ is a corank-1 point of ϕ , then

$A^{-1}\hat{u}$ is the corresponding corank-1 point of ψ . Moreover, the canonical form induced by such corresponding points is preserved, namely

$$\text{CF}(\phi, \hat{u}) = \text{CF}(\psi, A^{-1}\hat{u}).$$

Therefore, $\text{CF}(\phi, \hat{u})$ can be regarded as an invariant associated with the point $\hat{u}$. The attacker constructs two lists of invariants. First, sample a set of corank-1 points $\hat{u}$ from ϕ and compute $\text{CF}(\phi, \hat{u})$ for each of them. Similarly, sample a set of corank-1 points $\hat{u}'$ from ψ and compute $\text{CF}(\psi, \hat{u}')$. If the invariant is distinguishing, then different corank-1 points give different canonical forms with high probability. Hence, a collision

$$\text{CF}(\phi, \hat{u}) = \text{CF}(\psi, \hat{u}')$$

is expected to reveal a pair of corresponding points under the secret equivalence. Once such a collision is found, the problem is reduced to a linear-algebraic recovery step. Indeed, the computations of $\text{CF}(\phi, \hat{u})$ and $\text{CF}(\psi, \hat{u}')$ produce transformations sending ϕ and ψ to the same canonical representative. By composing the transformation from ϕ to this canonical form with the inverse of the transformation from ψ to the same canonical form, one obtains an equivalence (A, B, C) between ϕ and ψ . For a random trilinear form, the number of corank-1 points is approximately q^{n-2} . Hence, by the birthday paradox, it is enough to construct two lists of size about $q^{(n-2)/2}$. The resulting attack has complexity

$$O(q^{(n-2)/2} \cdot (qn^3 + n^4) \cdot (\log q)^2),$$

where the main cost comes from sampling corank-1 points, computing their induced canonical forms, and searching for a collision between the two lists. Compared with directly solving the full algebraic system, this method avoids the direct Gröbner basis computation and instead exploits the distinguishing power of point-induced canonical forms.

Narayanan, Qiao, and Tang [NQT24] also observe that the additional factor q in the point-sampling cost may potentially be avoided. Once one effective corank-one point has been found, the corank-one points encountered along its tripartite path may be reused as starting points for further invariant computations. This leads to the improved heuristic estimate

$$O(q^{(n-2)/2} n^4 (\log q)^2).$$

The original work leaves open whether the points obtained in this way are distributed sufficiently close to uniformly over the effective corank-one locus. For parameter selection, TRINE takes the conservative approach and assumes that this improvement is available to the attacker.

Hull-based attack. Couvreur and Levrat [CL25] proposed a different algorithm for the general Matrix Code Equivalence problem. A trilinear form can be represented by the matrix code spanned by its slices in one coordinate; when these slices are linearly independent, equivalence of the trilinear forms induces equivalence of the corresponding matrix codes.

The attack first passes, when advantageous, to the dual matrix code. It then reduces matrix-code equivalence to a matrix-code conjugacy problem by guessing a pair of corresponding matrices in the dual codes. For the resulting conjugate matrix codes, the attacker considers their hulls with respect to a trace bilinear form. When the hull is

one-dimensional, a normalized characteristic polynomial of a hull generator provides an invariant suitable for a list-collision search. A collision identifies conjugate hull generators, after which the full matrix-code equivalence can be recovered by polynomial-system solving or linear-algebraic methods.

For a k -dimensional code of $m \times n$ matrices, the attack applies to a broad range of possibly unbalanced parameters. In the $m = n = k$ case used by TRINE, Couvreur and Levrat obtain the time complexity

$$\tilde{O}(q^{n/2})$$

field operations. This has the same dominant exponent in q as the original analysed complexity of the Narayanan–Qiao–Tang attack.

The claim that the two attacks have the same asymptotic running time refers to the original Narayanan–Qiao–Tang analysis, in which sampling a fresh corank-one point costs an additional factor q . It does not include the heuristic sampling improvement described above. Consequently, for the TRINE parameter sets we use the minimum of the concrete hull-based estimate and the more conservative, factor q -free estimate for the corank-one invariant attack.

Rare structure attack Another relevant attack is the rare-structure attack of Ran and Samardjiska [RS24]. This attack considers the tripartite graph naturally associated with a trilinear form and searches for a triangle, namely a triple of projective points $(\hat{u}, \hat{v}, \hat{w})$ such that the trilinear form vanishes after fixing any two of them. For a random trilinear form, the expected number of such triangles is about $1/q$, namely with probability about $1/q$ the graph contains a unique triangle. If unique triangles are found in two equivalent trilinear forms, they must correspond under the hidden isometry. This gives partial information on the three secret invertible matrices, after which the remaining equivalence can be recovered by a polynomial-time post-processing step.

The main cost of the attack is therefore the algebraic search for triangles. Ran and Samardjiska model the triangle conditions as a system of quadratic equations and solve it using Gröbner-basis techniques adapted to tri-graded polynomial systems. This attack applies only to the fraction of keys whose graph contains such a rare triangle, but for those keys it can be significantly faster than directly solving the full system. Since the cost of this algebraic step grows mainly with the tensor dimension, the choice of the TRINE dimension n is made so that the estimated cost of this attack, together with the corank-one invariant attack and the hull-based attack, remains above the target security level.

6 Parameter choices

Our parameter choices are based on the following considerations.

1. Let λ be the target security level.
2. Select the dimension n based on the rare-structure attack in Section 5.3.
3. Select the field order q based on the invariant attack in Section 5.3.
4. Select the round number r , the unbalanced parameter K , and the form number in each round X based on Section 3.4.

The choices of n and q are coupled. Rare structure attacks are mostly controlled by n , while invariant collision attacks scale with powers of q whose exponents depend on n . Moreover, the rare-triangle attack applies to an approximately $1/q$ fraction of random instances, so we record both its conditional cost and its occurrence-adjusted cost.

6.1 The choices of n

The dimension n is primarily chosen to withstand algebraic attacks whose complexity is mainly governed by the dimension rather than by the field size. The most relevant attack of this type is the rare-triangle attack of Ran and Samardjiska [RS24] described at Section 5.3. This attack searches for a triangle in the tripartite graph naturally associated with a trilinear form. The dominant cost of the attack is the algebraic triangle-finding step. This step is modeled as a system of quadratic equations and analysed using tri-graded XL. The estimates in [RS24] include two relevant quantities. The first is the cost predicted by the solving degree. The second is the cost associated with the first degree fall. Since degree falls may significantly speed up practical Gröbner basis computations, TRINE uses the first-degree-fall estimate as the attacker-friendly estimate when choosing n . This gives a conservative choice of the dimension n .

In Table 5, we require the occurrence-adjusted first-degree-fall estimate to exceed the target security level. We also report the conditional first-degree-fall cost, because it describes the complexity of attacking a key conditioned on the rare structure being present.

Parameter set	n	q	1st.fall.deg-compl.	inc.search
TRINE-128	22	4093	130	142
TRINE-256	40	4093	248	260
TRINE-512	81	4093	515	527

Table 5: Dimension selection based on the rare-triangle attack.

For completeness, we also provide the corresponding solving-degree estimates at Table 6. These are not used as the primary criterion for selecting n , but they provide an upper-estimate comparison for the algebraic triangle-finding step.

Parameter set	n	q	Sol.deg-compl.	inc.search
TRINE-128	22	4093	143	155
TRINE-256	40	4093	261	273
TRINE-512	81	4093	529	541

Table 6: Dimension selection based on the rare-triangle attack.

Quantum attacks. For quantum security estimates, we apply the standard cubic speed-up for collision-search attacks based on quantum walks. Thus the corank-one invariant attack with a classical list size $q^{(n-2)/2}$ is estimated as

$$\tilde{O}(q^{(n-2)/3})$$

up to polynomial factors. The corresponding quantum estimates are reported separately in Table 7

6.2 The choices of q

After choosing a suitable range for n , the field size q is further selected based on the best known invariant-based collision attacks. Increasing q increases the cost of these attacks, but also increases the public key and signature sizes linearly in $\log_2 q$.

Parameter set	n	quantum.compl.
TRINE-128	22	105
TRINE-256	40	180
TRINE-512	81	348

Table 7: Dimension selection based on the quantum attack.

The relevant attacks are the corank-one invariant attack of Narayanan, Qiao, and Tang [NQT24] and the hull-based algorithm of Couvreur and Levrat [CL25] introduced in Section 5.3. TRINE takes the conservative approach and use the estimate

$$O(q^{(n-2)/2}n^4(\log q)^2).$$

For each parameter set, the estimates are summarized in Table 8

Parameter set	n	q	compl.
TRINE-128	22	4093	145
TRINE-256	40	4093	256
TRINE-512	81	4093	512

Table 8: Dimension selection based on the invariant-based attack.

6.3 The choices of X , r and K

We use the unbalanced challenge technique as in Section 9. This relies on three parameters, the round number r , the unbalanced parameter K , and the form number in each round X . To achieve λ -bits security, we require that $\binom{r}{K} \cdot X^K \geq 2^\lambda$. Table illustrates the bit securities of our choices of r , K , and X .

parameter set	r	K	X	security level of TRINE (bit)
TRINE-128	138	52	1	128.0
	61	36	4	128.2
TRINE-256	271	104	1	256.0
	116	76	4	256.1
TRINE-512	534	211	1	512.1
	232	149	4	512.0

Table 9: The bit security of TRINE for the choices of X , r and K .

References

- BBC⁺20. Magali Bardet, Maxime Bros, Daniel Cabarcas, Philippe Gaborit, Ray Perlner, Daniel Smith-Tone, Jean-Pierre Tillich, and Javier Verbel. Improvements of algebraic attacks for solving the rank decoding and minrank problems. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 507–536. Springer, 2020.
- BDN⁺23. Markus Bläser, Dung Hoang Duong, Anand Kumar Narayanan, Thomas Plantard, Youming Qiao, Arnaud Sipasseuth, and Gang Tang. The alteq signature scheme: Algorithm specifications and supporting documentation. *NIST PQC Submission*, 2023.
- BFV13. Charles Bouillaguet, Pierre-Alain Fouque, and Amandine Véber. Graph-theoretic algorithms for the “isomorphism of polynomials” problem. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 211–227. Springer, 2013.
- BMPS20. Jean-François Biasse, Giacomo Micheli, Edoardo Persichetti, and Paolo Santini. Less is more: code-based signatures without syndromes. In *International Conference on Cryptology in Africa*, pages 45–65. Springer, 2020.
- CL25. Alain Couvreur and Christophe Levrat. Highway to hull: An algorithm for solving the general matrix code equivalence problem. In *Annual International Cryptology Conference*, pages 253–283. Springer, 2025.
- CNP⁺23a. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Lars Ran, Tovohery Hajatiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Meds–matrix equivalence digital signature (2023). *Submission to the NIST Digital Signature Scheme standardization process*, 2023.
- CNP⁺23b. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Tovohery Hajatiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Take your meds: digital signatures from matrix code equivalence. In *International conference on cryptology in Africa*, pages 28–52. Springer, 2023.
- FS86. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- GMW91. Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in np have zero-knowledge proof systems. *Journal of the ACM (JACM)*, 38(3):690–728, 1991.
- GQ23. Joshua Grochow and Youming Qiao. On the complexity of isomorphism problems for tensors, groups, and polynomials i: tensor isomorphism-completeness. *SIAM Journal on Computing*, 52(2):568–617, 2023.
- GQ24. Joshua A Grochow and Youming Qiao. On p-group isomorphism: Search-to-decision, counting-to-decision, and nilpotency class reductions via tensors. *ACM Transactions on Computation Theory*, 16(1):1–39, 2024.
- GQT22. Joshua A Grochow, Youming Qiao, and Gang Tang. Average-case algorithms for testing isomorphism of polynomials, algebras, and multilinear forms. *journal of Groups, complexity, cryptology*, 14, 2022.
- HMR⁺10. Sean Hallgren, Cristopher Moore, Martin Rötteler, Alexander Russell, and Pranab Sen. Limitations of quantum coset states for graph isomorphism. *Journal of the ACM (JACM)*, 57(6):1–33, 2010.
- LQ17. Yinan Li and Youming Qiao. Linear algebraic analogues of the graph isomorphism problem and the erdős-rényi model. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 463–474. IEEE, 2017.
- NQT24. Anand Kumar Narayanan, Youming Qiao, and Gang Tang. Algorithms for matrix code and alternating trilinear form equivalences via new isomorphism invariants. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 160–187. Springer, 2024.
- RS24. Lars Ran and Simona Samardjiska. Rare structures in tensor graphs: Bermuda triangles for cryptosystems based on the tensor isomorphism problem. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 66–96. Springer, 2024.
- TDJ⁺22. Gang Tang, Dung Hoang Duong, Antoine Joux, Thomas Plantard, Youming Qiao, and Willy Susilo. Practical post-quantum signature schemes from isomorphism problems of trilinear forms. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 582–612. Springer, 2022.